Homework 4

Collaboration with other students are encouraged; remember to name anyone you collaborate with and to write up your final solution by yourself.

When writing proofs, mathematical clarity is just as important as correctness and rigor. Please keep this in mind and try to adhere to proof writing conventions when typing up solutions.

1. Prove constructively that evaluations at $d+1$ distinct points uniquely determine a polynomial of degree at most d.

   Let $x_0, x_1, \ldots, x_d$ be distinct points, and suppose we are given values

   $$f(x_i) = y_i \quad \text{for } i = 0, 1, \ldots, d,$$

   $f(x)$ some polynomial with degree at most d.

   (a) For each i, explicitly construct a polynomial $L_i(x)$ of degree at most d such that

   $$L_i(x_j) = \begin{cases} 1 & \text{if } j = i, \\ 0 & \text{if } j \neq i. \end{cases}$$

   Show that your construction satisfies the constraints of the problem.

   (b) Using your polynomials from part (a), define

   $$p(x) = \sum_{i=0}^{d} y_i L_i(x).$$

   Show that $p(x_j) = y_j$ for all j.

   (c) Finally, prove that this polynomial is unique by showing that if $q(x)$ is any other polynomial of degree at most d satisfying $q(x_i) = y_i$ for all i, then $q(x) = p(x)$.

2. Consider the following divide-and-conquer algorithm for evaluating a degree-d polynomial

   $$p(x) = a_0 + a_1 x + \cdots + a_d x^d$$

   at $n = d + 1$ distinct points $\omega_0, \omega_1, \ldots, \omega_{n-1}$, where n is a power of two.

   We split the polynomial into its even and odd parts:

   $$p(x) = p_{\text{even}}(x^2) + x\, p_{\text{odd}}(x^2),$$

   where

   $$p_{\text{even}}(x^2) = a_0 + a_2 x^2 + a_4 x^4 + \cdots, \qquad x p_{\text{odd}}(x^2) = a_1 x + a_3 x^3 + a_5 x^5 + \cdots$$

   and thus,

   $$p_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots, \qquad p_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots$$

Algorithm: EvaluatePolynomial

Eval $([a_0, \ldots, a_{n-1}], [\omega_0, \omega_1, \ldots, \omega_{\frac{n}{2}-1}, -\omega_0, -\omega_1, \ldots, -\omega_{\frac{n}{2}-1}])$**:**

- **if**
  - $n \leq 1 \rightarrow$ **return** (a_0)
  - $n > 1 \rightarrow$
    - $y^{(e)} := \text{Eval}(a_{\text{even}}, \omega_0^2, \ldots, \omega_{n/2-1}^2)$
    - $y^{(o)} := \text{Eval}(a_{\text{odd}}, \omega_0^2, \ldots, \omega_{n/2-1}^2)$
- $k := 0$
- **do**
  - $k < n/2 \rightarrow$
    - $y_k := y_k^{(e)} + \omega_k \cdot y_k^{(o)}$
    - $y_{k+n/2} := y_k^{(e)} - \omega_k \cdot y_k^{(o)}$
    - $k := k + 1$
- **return** $(y_0, \ldots, y_{n-1})$

(a) Let $T(n)$ denote the running time of `Eval` on an input of size n. Write a recurrence and solve it to analyze the runtime of the algorithm.

(b) Suppose we restrict all evaluation points to be real numbers, and we attempt to run `Eval` with $n = 4$.

Consider the polynomial with coefficients

$$(a_0, a_1, a_2, a_3) = (1, 2, 3, 4),$$

and choose the evaluation points

$$(\omega_0, \omega_1, \omega_2, \omega_3) = (1, -1, 1, -1),$$

(i) Compute a_{even} and a_{odd}.

(ii) Compute the squared points used by the recursive calls:

$$(\omega_0^2, \omega_1^2) \quad \text{and} \quad (\omega_2^2, \omega_3^2).$$

(iii) Trace `Eval` by hand on this input (show the values returned by the two recursive calls and then show the four outputs produced by the combine loop).

(iv) Separately compute the true evaluations $p(\omega_k)$ for $k = 0, 1, 2, 3$ and compare with the algorithm's output. What structural problem do you observe with the chosen real evaluation points?

(c) Explain why the failure in part (b) is not an accident: argue that for $n > 2$ there are not enough distinct real n-th roots of unity to support this divide-and-conquer strategy by answering the following parts:

(i) Why does the algorithm rely on having n distinct points with the symmetry property $\omega_{k+n/2} = -\omega_k$ and the closure property that squaring maps the first half of the points onto the points used in the recursive call (doesn't need to be a proof but answers should have some math to back up answer)?

(ii) Prove the correctness of the `Eval` algorithm when the evaluation points are chosen to be the n-th roots of unity in $\mathbb{C}$ through induction.

Note: a_{even} are all even-indexed coefficients $(a_0, a_2, a_4, \ldots)$ and a_{odd} are all odd-indexed coefficients $(a_1, a_3, a_5, \ldots)$.

3. Given n coins which each have their own probability of landing heads $c_1, c_2, \ldots, c_n$, we are interested in analyzing the probability that exactly k of the flips landed on heads. Design an efficient *divide-and-conquer* algorithm that performs the task above:

   Input: Coins with probabilities $\mathcal{C} = \{c_1, c_2, \ldots, c_n\}$ of landing heads, and some integer $0 \leq k \leq n$
   Output: The probability exactly k coins landed heads

   (a) First try a simple case by hand. If there are 3 coins, with probabilities c_1, c_2, and c_3 of landing on heads, what is the probability that when we flip all of them exactly 2 of them will be heads?
   Hint: Your answer should be written in terms of c_1, c_2, and c_3.

   (b) For each coin c_i, what is a polynomial function $p_i(x)$ such that when you compute

   $$p(x) = \prod_{i=1}^{n} p_i(x)$$

   the coefficient of the x^kth term of $p(x)$ is the probability that exactly k flips landed on heads?

   (c) Use the parts above to describe an algorithm which solves the problem. Be sure to justify its correctness.

   (d) Provide a recurrence for your algorithm and use it to analyze the runtime.
   Hint: To solve the recurrence, It's easier to divide both sides by n and to try analyzing $\frac{T(n)}{n}$.